



BACKEND INTERVIEW HANDBOOK



MOST IMPORTANT CONCEPTS & QUESTIONS

1/10

1. BACKEND FUNDAMENTALS

1. What is Backend Development?

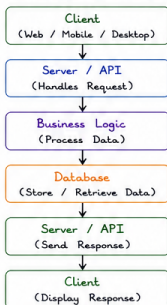
Backend development refers to server-side development. It involves building the logic, database, APIs and server infrastructure that powers the application. The backend is **not visible** to users.

Key Responsibilities

- Handle business logic
- Manage databases
- Authentication & Authorization
- Build and expose APIs
- Ensure security
- Optimize performance & scalability



2. Client → Server → Database Flow



Flow Explanation

- ① Client sends a request to the server.
- ② Server validates the request.
- ③ Business logic is executed.
- ④ Data is read from / written to database.
- ⑤ Response is returned to the server.
- ⑥ Server sends response back to client.

3. Stateless vs Stateful

Stateless	Stateful
• Server does not store client context. • Each request is independent. • Easier to scale. • Examples: REST APIs, JWT Authentication.	• Server stores client context (session). • Requests depend on previous state. • Harder to scale. • Examples: Traditional sessions, carts.



Key Point: Stateless systems are preferred in modern backend (microservices, APIs).

4. Monolith vs Microservices

Monolith	Microservices
Single codebase for entire application.	Application is broken into small, independent services.
Easy to develop initially.	Harder to develop initially.
Difficult to scale.	Easy to scale individual services.
Deployment is simple.	Deployment is complex.
Example: Traditional web applications.	Example: Netflix, Uber, Amazon.

5. REST vs SOAP

REST	SOAP
Architectural style	Protocol
Uses HTTP methods (GET, POST, PUT, DELETE)	Uses XML messaging
Lightweight	Heavyweight
Easy to understand	Complex
Better performance	Lower performance
Supports JSON, XML, Text, etc.	Supports only XML
Widely used in modern applications	Used in enterprise applications

6. Common Backend Interview Questions

- ① What is the difference between REST and SOAP?
- ② What is the difference between Stateful and Stateless?
- ③ What is the purpose of an API?
- ④ How does a web request flow from client to database?
- ⑤ What are the advantages of microservices?
- ⑥ What is the difference between PUT and PATCH?
- ⑦ What is idempotency in HTTP methods?
- ⑧ What are the common status codes in HTTP?
- ⑨ What is the role of middleware?
- ⑩ How do you handle errors in an API?

Interview Tip



Speak clearly, give real-life examples and explain concepts with diagrams if possible.



Remember

Strong Fundamentals → Clear Concepts → Real-world Examples = Successful Interview



2. HTTP & REST APIs

1. HTTP Methods

Method	Purpose	Idempotent	Safe
GET	Retrieve data	✓	✓
POST	Create new resource	✗	✗
PUT	Update entire resource	✓	✗
PATCH	Update partial resource	✓	✗
DELETE	Delete resource	✓	✗

- **Safe:** Method does not modify the resource.
- **Idempotent:** Multiple calls have the same effect as a single call.

3. REST Principles

- **Client-Server Architecture:**
- **Stateless:** Server does not store client state.
- **Cachable:** Responses can be cached.
- **Uniform Interface:** Consistent API design.
- **Layered System:** Components can be layered.
- **Code on Demand (optional):** Server can send executable code.
- **Resources are identified using URIs.**
- **Use HTTP methods to perform operations on resources.**



4. URI Best Practices

- Use nouns, not verbs.
Good: /users **Bad:** /getUsers
- Use lowercase letters.
- Use hyphens (-) to separate words.
Good: /user-profile **Bad:** /userProfile
- Use hierarchical structure.
Good: /users/123/posts **Bad:** /posts/123/byUser
- Use plural nouns for resources.
Good: /users **Bad:** /user
- Keep URIs simple and consistent.

Good Example

```

GET    /users
GET    /users/123
POST   /users
PUT    /users/123
DELETE /users/123
GET    /users/123/posts
POST   /users/123/posts
  
```

2. HTTP Status Codes (Common)

Category	Code	Meaning	Examples
1xx (Informational)	100	Continue	Request received, continue process.
	200	OK	Request successful.
2xx (Success)	201	Created	Resource created successfully.
	204	No Content	Request successful, no content.
	301	Moved Permanently	Resource moved permanently.
3xx (Redirection)	304	Not Modified	Resource not modified.
	400	Bad Request	Invalid request.
	401	Unauthorized	Authentication required.
	403	Forbidden	Access denied.
4xx (Client Error)	404	Not Found	Resource not found.
	500	Internal Server Error	Something went wrong.
	503	Service Unavailable	Server is overloaded.

5. RESTful API Best Practices

- ✓ Use meaningful resource names.
- ✓ Use appropriate HTTP methods.
- ✓ Return proper status codes.
- ✓ Keep requests & responses consistent.
- ✓ Use pagination, filtering, sorting.
- ✓ Version your APIs (e.g., /api/v1/users).
- ✓ Use HTTPS for secure communication.
- ✓ Validate input and handle errors properly.
- ✓ Follow principle of least privilege.
- ✓ Document your APIs (Swagger / OpenAPI).

Example Response

```

{
  "status": "success",
  "data": {
    "id": 123,
    "name": "John Doe",
    "email": "john@example.com"
  },
  "message": "User retrieved",
  "timestamp": "2024-05-12T10:00:00Z"
}
  
```

6. Common Interview Questions & Answers

Q1: What is REST?

A: REST (Representational State Transfer) is an architectural style for designing networked applications.

Q2: What is the difference between PUT and PATCH?

A: PUT updates the entire resource, PATCH updates partial fields of the resource.

Q3: What is idempotency?

A: Multiple identical requests have the same effect as a single request. (e.g., PUT, DELETE)

Q4: What is the difference between 200 and 201?

A: 200 OK is returned for a successful request. 201 Created is returned when a new resource is created.

Q5: What is the difference between 401 and 403?

A: 401 Unauthorized means authentication is required. 403 Forbidden means authenticated but not allowed.

Q6: What is stateless in REST?

A: Server does not store any client state. Each request from client must contain all information needed.

Q7: What is a resource in REST?

A: Resource is any object or data that can be identified using a URI.

Q8: Why do we use HTTP methods in REST APIs?

A: HTTP methods define the type of operation to be performed on the resource.



Interview Tip

Explain with simple examples, draw diagrams if needed and relate concepts to real-world scenarios.





4. DATABASES

1. SQL vs NoSQL

SQL (Relational)	NoSQL (Non-Relational)
Structured schema	Flexible schema
Table-based	Document / Key-Value / Column / Graph
ACID compliant	BASE (Mostly)
Joins supported	Joins not common
Best for complex queries	Best for large scale & unstructured data
Examples: MySQL, PostgreSQL, SQL Server	Examples: MongoDB, Redis, Cassandra, DynamoDB



Key Point: Choose SQL for relational data with transactions. Choose NoSQL for scalability and high availability.

2. Keys

- Primary Key:** Uniquely identifies each record in a table.
- Foreign Key:** Refers to the primary key in another table.
- Composite Key:** Combination of two or more columns to uniquely identify a record.



Tip: Always index foreign keys for better performance.

3. Indexing

- Indexes improve the speed of data retrieval operations.
- Created on one or more columns.
- Types:
 - Primary Index (Clustered)
 - Secondary Index (Non-Clustered)
 - Unique Index
 - Composite Index

Best Practices

- ✓ Index columns queried frequently.
- ✓ Avoid over-indexing (slows down writes).
- ✓ Use composite index for multiple columns.
- ✓ Regularly analyze and rebuild indexes.

4. Normalization (Up to 3NF)

- 1NF:** Eliminate repeating groups. Ensure atomic values.
- 2NF:** Remove partial dependency. Non-key columns fully depend on the primary key.
- 3NF:** Remove transitive dependency. Non-key columns should not depend on other non-key columns.

Example:



Tip: Follow normalization to reduce data redundancy and maintain data integrity.

5. ACID Properties

Property	Description
Atomicity	All operations in a transaction are completed successfully or none.
Consistency	Transaction brings the database from one valid state to another.
Isolation	Transactions are isolated from each other until completion.
Durability	Once committed, data is permanent even in case of failure.

Example Scenario

Money Transfer:
 • Deduct from A
 • Add to B
 If any step fails, transaction rolls back.



Key Point: ACID ensures reliable and consistent transactions in relational databases.

6. Transactions

- A transaction is a sequence of operations performed as a single logical unit.
- Either all operations succeed or all fail.
- Managed using BEGIN, COMMIT, ROLLBACK.

BEGIN TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;

UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT; -- if success

-- ROLLBACK; -- if any error



Tip: Keep transactions short to avoid locks and improve concurrency.

7. Joins in SQL

Join Type	Description
INNER JOIN	Returns matching rows in both tables.
LEFT JOIN	Returns all rows from left table and matches from right table.
RIGHT JOIN	Returns all rows from right table and matches from left table.
FULL JOIN	Returns all rows when there is a match in either table.

INNER JOIN



LEFT JOIN



RIGHT JOIN



FULL JOIN



8. Common SQL Questions

- What is the difference between WHERE and HAVING?
- What is the difference between DELETE, TRUNCATE and DROP?
- What are GROUP BY and ORDER BY?
- What is a subquery?
- What is the difference between UNION and UNION ALL?
- What is a stored procedure?
- What is the use of an index?
- What is a view?
- What is a deadlock?
- What is a transaction?

9. Quick Interview Notes

SQL Commands

- DDL: CREATE, ALTER, DROP
- DML: SELECT, INSERT, UPDATE, DELETE
- DCL: GRANT, REVOKE
- TCL: COMMIT, ROLLBACK, SAVEPOINT

Useful Functions

- COUNT() - count rows
- SUM() - sum values
- AVG() - average value
- MIN() / MAX() - min / max value
- UPPER() / LOWER() - case change
- NOW() - current date & time

Aggregate vs Non-Aggregate

- Aggregate: COUNT, SUM, AVG, MIN, MAX
- Non-Aggregate: Column names
- Rule: If you use aggregate function, all non-aggregate columns must be in GROUP BY.



Interview Tip

Practice writing queries on paper before running them. Always check for performance and edge cases.

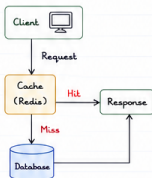




5. PERFORMANCE OPTIMIZATION

1. Caching

- Caching stores frequently used data in memory so it can be served faster.
- Reduces database load and improves response time.
- Types: Application Cache, Distributed Cache (Redis), CDN Cache.



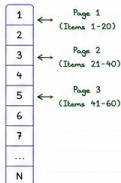
Tip: Cache what is expensive to compute or frequently read, but not what changes often.

2. Pagination

- Pagination breaks large result sets into smaller chunks (pages).
- Improves performance and reduces memory usage.
- Use Limit & Offset or Cursor based pagination.

Example (Limit & Offset)

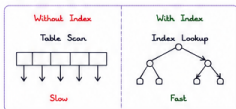
```
SELECT * FROM users
ORDER BY id
LIMIT 20 OFFSET 40; -- Returns 3rd page (size 20)
```



Best Practice: Always set a max page size to prevent heavy queries.

3. Database Indexing

- Indexes improve query performance by reducing search time.
- Created on commonly used columns in WHERE, JOIN, ORDER BY.
- Types: Primary Index, Secondary Index, Composite Index.



Tip: Too many indexes slow down write operations. Use only where needed.

4. Query Optimization

- Write efficient queries.
- Avoid SELECT *.
- Use proper indexes.
- Avoid functions on indexed columns.
- Use EXPLAIN to analyze query plan.

Bad Query:

```
SELECT * FROM orders
WHERE YEAR(created_at) = 2024;
```

Good Query:

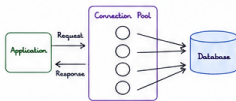
```
SELECT * FROM orders
WHERE created_at >= '2024-01-01'
AND created_at < '2025-01-01';
```



Tip: Always check the execution plan and optimize bottlenecks.

5. Connection Pooling

- Reuses existing database connections instead of creating new ones.
- Reduces connection overhead.
- Improves performance and scalability.
- Common Pools: HikariCP, DBCP, C3PO.



Tip: Set proper pool size based on DB capacity and application load.

6. Lazy Loading vs Eager Loading

	Lazy Loading	Eager Loading
Definition	Load data only when it is needed.	Load all related data in advance.
Performance	Better for large data with selective access.	Better when related data is always needed.
Database Calls	Multiple (N+1 problem possible)	Single (Uses JOIN)
Use When	Data is large and not always required.	Data is small or always required together.



Tip: Avoid N+1 problem in lazy loading using JOIN or Batch Fetching.

7. Compression

- Compress response data to reduce payload size.
- Common algorithms: Gzip, Brotli.
- Saves bandwidth and improves response time.

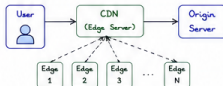
Content-Encoding: gzip
Accept-Encoding: gzip, deflate, br



Tip: Enable compression for text, JSON, XML responses.

8. CDN (Content Delivery Network)

- CDN stores static content (images, CSS, JS, videos) in edge servers.
- Reduces latency and server load.
- Examples: Cloudflare, AWS CloudFront, Akamai.



Tip: Use CDN for static assets and files that rarely change.

9. Other Best Practices

- Use asynchronous processing for long running tasks.
- Implement rate limiting to prevent server overload.
- Use proper logging and monitoring.
- Avoid heavy operations in loops.
- Use background jobs/message queues for heavy tasks.

- Keep dependencies up to date.
- Use load balancer to distribute traffic.
- Regularly profile and monitor performance.
- Design for scalability from the beginning.



Interview Tip

Always analyze the bottleneck first, then optimize. Measure before and after optimization.



Optimize Smartly. Deliver Fast. Scale Confidently.

Performance is not an accident, it is the result of good design and continuous improvement.





6. SYSTEM DESIGN BASICS

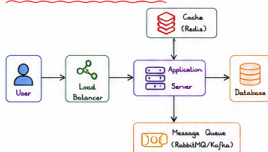
1. What is System Design?

- System Design is the process of designing scalable, reliable, and efficient systems to meet requirements.
- Focus: Scalability, Availability, Performance, Reliability, Maintainability.

Key Metrics

- ✓ Scalability - Handle growth
- ✓ Availability - Minimize downtime
- ✓ Reliability - Fault tolerant
- ✓ Performance - Fast response time
- ✓ Maintainability - Easy to update

2. High Level Architecture



Flow

1. User sends request.
2. Load Balancer distributes request.
3. Application Server processes request.
4. Cache is checked for quick response.
5. Data is read/written to Database.
6. Message Queue handles async tasks.

3. Load Balancer

- Distributes incoming traffic across multiple servers.
- Types: Hardware, Software.
- Algorithms:
 - Round Robin
 - Least Connections
 - IP Hash
 - Weighted Round Robin

Benefits

- ✓ High Availability
- ✓ Scalability
- ✓ Better Performance
- ✓ Fault Tolerance

4. Database Replication

- Replication keeps multiple copies of data across different servers.
- Types:
 - Master-Slave Replication
 - Master-Master Replication
 - Multi-Region Replication

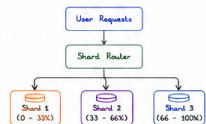


Benefits

- ✓ High Availability
- ✓ Read Scalability
- ✓ Disaster Recovery

5. Sharding

- Sharding is a technique to split data horizontally across multiple databases.
- Improves scalability and performance.
- Example: User table sharded by UserID.



Best Practice

- ✓ Choose shard key carefully.
- ✓ Avoid cross-shard joins.
- ✓ Rebalance shards as data grows.

6. Caching

- Caching stores frequently used data in memory for faster access.
- Reduces load on database.
- Example: Redis, Memcached.



Best Practice

- ✓ Cache what is expensive to compute.
- ✓ Set proper TTL (Time to Live).
- ✓ Use cache invalidation strategy.

7. Message Queue

- Used for async communication between services.
- Decouples services and improves scalability.
- Examples: RabbitMQ, Kafka, SQS.



Benefits

- ✓ Asynchronous Processing
- ✓ Improved Scalability
- ✓ Fault Tolerance
- ✓ Load Leveling

8. Microservices Architecture

- Application is divided into small, independent services.
- Each service has its own database.
- Communicate via APIs or Message Queue.



Benefits

- ✓ Scalability
- ✓ Technology Freedom
- ✓ Independent Deployment
- ✓ Fault Isolation

9. Scalability Strategies

Vertical Scaling (Scale Up)	Increase power of a single server.
Horizontal Scaling (Scale Out)	Add more servers to distribute load.
Auto Scaling	Automatically add/remove servers based on traffic.
Database Sharding	Split data across multiple databases.
Caching	Reduce load and improve response time.
CDN	Deliver content closer to users.

General Best Practices

- ☑ Design for failure.
- ☑ Keep components loose coupled.
- ☑ Monitor and log everything.
- ☑ Use proper security (HTTPS, Auth, Authorization).
- ☑ Optimize for common case.
- ☑ Plan for growth from day one.
- ☑ Think: Can it handle 10x traffic?

Remember

A good system design balances scalability, performance, and simplicity.



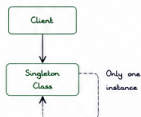
7. DESIGN PATTERNS



Design Patterns are reusable solutions to common problems in software design.

1. Singleton Pattern

- Ensures a class has only one instance and provides a global point of access to it.
- Useful for logging, configuration, database connection.

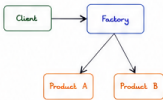


Use Cases

- ✓ Configuration Manager
- ✓ Logger
- ✓ Database Connection

2. Factory Pattern

- Provides an interface for creating objects but allows subclasses to alter the type of objects that will be created.
- Promotes loose coupling.



Use Cases

- ✓ When object creation logic is complex
- ✓ When we want to add new types without changing client code

3. Repository Pattern

- Acts as a mediator between the domain and data mapping layers.
- Hides data access logic from business logic.



Use Cases

- ✓ Data access abstraction
- ✓ Easy unit testing (mock repository)
- ✓ Switching data source

4. Strategy Pattern

- Defines a family of algorithms, encapsulates each one, and makes them interchangeable.
- Enables algorithm to vary independently from clients.

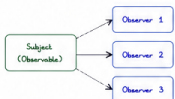


Use Cases

- ✓ Multiple algorithms for a task
- ✓ Different payment methods
- ✓ Different sorting strategies

5. Observer Pattern

- Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified.
- Useful for event handling.



Use Cases

- ✓ Event handling systems
- ✓ Notifications
- ✓ Real-time updates

6. Dependency Injection (DI) Pattern

- A way to achieve Inversion of Control (IoC).
- Dependencies are provided from outside rather than created inside the class.

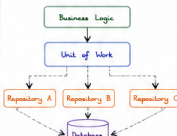


Use Cases

- ✓ Loosely coupled applications
- ✓ Easy testing & mocking
- ✓ Scalable & maintainable code

7. Unit of Work Pattern

- Maintains a list of objects affected by a business transaction.
- Ensures all changes are committed together or rolled back together.

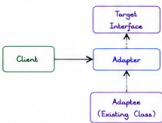


Use Cases

- ✓ Transaction management
- ✓ Complex business operations
- ✓ Consistency across operations

8. Adapter Pattern

- Converts the interface of a class into another interface that clients expect.
- Helps classes with incompatible interfaces work together.



Use Cases

- ✓ Integrating third-party libraries
- ✓ Legacy system integration
- ✓ Interface incompatibility

SOLID Principles (Key to Good Design)

S	Single Responsibility Principle	A class should have only one reason to change.
O	Open/Closed Principle	Classes should be open for extension but closed for modification.
L	Liskov Substitution Principle	Objects of a superclass should be replaceable with objects of its subclasses.
I	Interface Segregation Principle	Clients should not be forced to depend on interfaces they do not use.
D	Dependency Inversion Principle	High-level modules should not depend on low-level modules. Both should depend on abstractions.

Why SOLID?

- ✓ Improves code maintainability
- ✓ Reduces code complexity
- ✓ Easier to test, extend and refactor
- ✓ Follows best OOP practices

Example: Dependency Inversion

```

// High-level module depends on abstraction
public class UserService {
    private readonly IUserRepository _repo;
    public UserService(IUserRepository repo) {
        _repo = repo;
    }
}

```

```

// Abstraction
public interface IUserRepository {
    User GetById(int id);
}

// Low-level module
public class SqlUserRepository : IUserRepository {
    public User GetById(int id) { ... }
}

```



Tip: Don't just memorize patterns. Understand the problem they solve and when to apply them.





8. JAVA BACKEND QUESTIONS

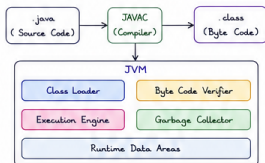
1. Java Basics

- Java is platform independent because of JVM.
- Follows OOP principles.
- Automatic memory management through Garbage Collection.
- Strongly typed and object oriented.

Key Concepts

- ✓ Class & Object
- ✓ Inheritance
- ✓ Polymorphism
- ✓ Abstraction
- ✓ Encapsulation

2. JVM Architecture



Runtime Data Areas:

Method Area, Heap Area, Java Stacks,
PC Register, Native Method Stack.

3. OOP Concepts

- Inheritance: Reusability of code.
- Polymorphism: One interface, multiple implementations.
- Abstraction: Hiding internal implementation.
- Encapsulation: Wrapping data and methods together.

Example - Polymorphism

```

class Animal {
    void sound() { System.out.println("Animal"); }
}
class Dog extends Animal {
    void sound() { System.out.println("Bark"); }
}
Animal a = new Dog();
a.sound(); // Bark
  
```

4. Collections Framework

- Provides data structures.
- Interfaces: List, Set, Queue, Map
- Classes: ArrayList, LinkedList, HashSet, TreeSet, HashMap, TreeMap

Interface	Implementation Classes
List	ArrayList, LinkedList, Vector
Set	HashSet, LinkedHashSet, TreeSet
Queue	PriorityQueue, ArrayDeque
Map	HashMap, LinkedHashMap, TreeMap

Tip: Use ArrayList for fast access, HashMap for key-value lookups.

5. Exception Handling

- Handle runtime errors.
- Improves application stability.
- Keywords: try, catch, finally, throw, throws

```

try {
    int a = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("This always executes");
}
  
```

Tip: Prefer specific exceptions over general Exception.

6. Multithreading

- Allows concurrent execution.
- Extends Thread class or implements Runnable interface.
- Improves performance.

Extends Thread

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}
MyThread t = new MyThread();
t.start();
  
```

Tip: Use ExecutorService for better thread management.

7. Generics

- Provides type safety.
- Eliminates ClassCastException at runtime.
- Improves code reusability.

```

class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}
Box<String> box = new Box<>();
box.set("Hello");
String s = box.get();
  
```

Tip: Use bounded types when needed: <T extends Number>

8. Java 8 Features

- Lambda Expressions
- Stream API
- Functional Interfaces
- Optional class
- Method References

Lambda Example

```

List<String> names = Arrays.asList("A", "B", "C");
names.forEach(name -> System.out.println(name));
  
```

Stream Example

```

List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
int sum = list.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(Integer::intValue)
    .sum();
  
```

9. Important Java Concepts

- String vs StringBuilder vs StringBuffer
String (immutable),
StringBuilder (non-thread-safe),
StringBuffer (thread-safe)
- final: variable, method, class
- static: belongs to class
- this vs super
- == vs equals()
- Deep Copy vs Shallow Copy

10. Common Java Interview Questions

- What is JVM?
→ It is a runtime environment that executes Java bytecode.
- What is the difference between == and equals()?
→ == compares references, equals() compares content.
- What is a HashMap?
→ It stores key-value pairs and allows null key and multiple null values.
- What is the difference between ArrayList and LinkedList?
→ ArrayList is index based, LinkedList is node based.
- What is the use of static keyword?
→ To create class level variables and methods.
- What is synchronization?
→ It is used to control access of multiple threads to a resource.
- What is the difference between HashSet and TreeSet?
→ HashSet is unordered, TreeSet is sorted.
- What is a deadlock?
→ When two or more threads are blocked forever waiting for each other's resources.
- What is garbage collection?
→ Process of reclaiming memory by removing unused objects.
- What are functional interfaces?
→ Interfaces with single abstract method. (e.g. Runnable, Callable)





9. SCENARIO-BASED QUESTIONS (PRODUCTION-READY ANSWERS)

1. API is slow. What will you check?

- ✓ Check latency at each layer: Client → CDN → LB → App → DB.
- ✓ Check APM/Tracing (e.g., Jaeger/Zipkin) for slow endpoints.
- ✓ Review DB slow queries (enable slow query log).
- ✓ Check external service calls and timeouts.
- ✓ Verify cache hit ratio.
- ✓ Check server CPU, memory, thread count, GC.
- ✓ Check network latency and payload size.
- ✓ Look for N+1 queries and inefficient code.

Action:

Identify bottleneck → Optimize → Re-deploy → Monitor.

2. Database is overloaded.

- ✓ Check top slow queries and lock wait time.
- ✓ Check connections, CPU, Disk I/O, Buffer cache hit ratio.
- ✓ Add missing indexes and optimize queries.
- ✓ Scale read replicas for read-heavy workloads.
- ✓ Enable connection pooling.
- ✓ Partition large tables if needed.
- ✓ Move heavy/analytics queries to a separate DB.
- ✓ Consider caching frequently-read data.

Action:

Reduce load → Optimize queries → Scale DB → Monitor.

3. Cache is not updating.

- ✓ Check cache invalidation logic.
- ✓ Verify TTL (time to live) settings.
- ✓ Ensure write-through or write-back strategy is correct.
- ✓ Check if events/queues used for invalidation are working.
- ✓ Look for stale cache due to replication delay.
- ✓ Verify keys (uniqueness, versioning).

Action:

Fix invalidation → Update cache → Add versioning → Monitor.

5. High CPU usage.

- ✓ Check top CPU-consuming processes/threads.
- ✓ Profile the application (CPU profiler).
- ✓ Look for tight loops, inefficient algorithms.
- ✓ Check regex, JSON parsing, serialization hotspots.
- ✓ Optimize DB queries and reduce unnecessary work.
- ✓ Scale horizontally if the service is stateless.

Action:

Profile → Optimize code/queries → Scale → Monitor.

6. Deadlocks.

- ✓ Check DB deadlock logs and recent deadlock graph.
- ✓ Ensure consistent lock acquisition order.
- ✓ Keep transactions short.
- ✓ Add proper indexes to reduce locking.
- ✓ Use appropriate isolation levels.
- ✓ Implement retry with backoff on deadlock.

Action:

Fix query/txn logic → Add indexes → Retry → Monitor.

7. Large file upload optimization.

- ✓ Use pre-signed URLs and direct-to-storage uploads (S3/GCS).
- ✓ Use multipart upload for large files.
- ✓ Validate file type/size on client and server.
- ✓ Stream upload; avoid loading entire file in memory.
- ✓ Use CDN for download.
- ✓ Scan files async (virus scan, etc.) via queue.

Action:

Offload to object storage → Multipart upload → Async tasks.

8. How would you scale to 1M users?

- ✓ Use stateless services; scale horizontally behind a load balancer.
- ✓ Use CDN for static content.
- ✓ Cache aggressively (Redis/Memcached).
- ✓ Database: read replicas, sharding, indexing.
- ✓ Use connection pooling and async processing.
- ✓ Use message queues for heavy/background tasks.
- ✓ Design for idempotency and eventual consistency.
- ✓ Monitor, autoscale, set alerts.

Action:

Scale layers (Web → App → Cache → DB) + Observe.

General Tips



- ✓ Always monitor before you optimize.
- ✓ Measure, don't guess.
- ✓ Define SLAs and error budgets.
- ✓ Design for failure.
- ✓ Automate and test at scale.
- ✓ Keep it simple and observable.

Golden Rule



Find the bottleneck.
Fix the root cause.
Verify with metrics.
Prevent recurrence.

10. TOP 30 RAPID-FIRE QUESTIONS

1.		PUT vs PATCH?	→ PUT replaces the entire resource. PATCH updates partial fields.
2.		JWT vs Session?	JWT is stateless, self-contained. Session is stateful, stored on server.
3.		Redis vs Memcached?	Redis supports complex data types, persistence. Memcached is simple, in-memory key-value store.
4.		HashMap vs ConcurrentHashMap?	HashMap is not thread-safe. ConcurrentHashMap is thread-safe, highly concurrent.
5.		ACID vs BASE?	ACID: Strong consistency, transactions. BASE: Basically Available, Soft state, Eventual consistency.
6.		Authentication vs Authorization?	Authentication verifies who you are. Authorization determines what you can do.
7.		SQL vs NoSQL?	SQL is relational, structured, ACID. NoSQL is non-relational, flexible, scalable.
8.		Vertical vs Horizontal Scaling?	Vertical: Add more power to a server. Horizontal: Add more servers.
9.		Docker vs VM?	Docker shares OS kernel, lightweight. VM has full OS, heavier.
10.		Kafka vs RabbitMQ?	Kafka is high-throughput, pub/sub, log-based. RabbitMQ is message broker, flexible routing.
11.		Load Balancer vs API Gateway?	LB distributes traffic. API Gateway handles routing, auth, rate limiting, etc.
12.		CDN vs Cache?	CDN caches content at edge locations. Cache stores data closer to the application.
13.		Sync vs Async?	Sync waits for response. Async doesn't wait, continues execution.
14.		Thread vs Process?	Thread is lightweight, shares memory. Process is heavyweight, separate memory.
15.		Optimistic vs Pessimistic Locking?	Optimistic assumes no conflict. Pessimistic locks resources to prevent conflict.
16.		Clustered vs Non-Clustered Index?	Clustered index sorts & stores data. Non-clustered index has separate structure.
17.		Garbage Collection in Java?	Automatically reclaims unused memory. Types: Serial, Parallel, G1, etc.
18.		TCP vs UDP?	TCP is connection-oriented, reliable. UDP is connectionless, faster.
19.		Idempotent vs Non-Idempotent API?	Idempotent: Same result on repeated requests (e.g., PUT). Non-idempotent: Different result (e.g., POST).
20.		CAP Theorem?	Consistency, Availability, Partition tolerance. You can have any two.
21.		Polymorphism vs Abstraction?	Polymorphism: One interface, multiple forms. Abstraction: Hide complexity.
22.		Monolith vs Microservices?	Monolith: Single deployable. Microservices: Small, independent services.
23.		Circuit Breaker Pattern?	Prevents cascading failures. Opens circuit on failure, then half-open.
24.		Stateless vs Stateful Service?	Stateless doesn't store client state. Stateful stores client state.
25.		TTL in Cache?	Time To Live. Key expires after a given time.
26.		Replication vs Sharding?	Replication: Copy data. Sharding: Split data.
27.		Webhooks vs Polling?	Webhooks: Server pushes data. Polling: Client asks repeatedly.
28.		Symmetric vs Asymmetric Encryption?	Symmetric: Same key. Asymmetric: Public & private key pair.
29.		REST vs gRPC?	REST is HTTP/JSON based. gRPC is HTTP/2, Protocol Buffers, faster.
30.		Synchronous Blocking vs Asynchronous Non-Blocking?	Blocking waits for result. Non-blocking continues without waiting.

Tips

- ☑ Keep answers short, to the point.
- ☑ Focus on key differences.
- ☑ Give real-world examples if possible.
- ☑ Think out loud if unsure.

Remember

- ☑ Understand the concept, not just the definition.
- ☑ Relate answers to scalability, performance, and reliability.
- ☑ Practice these regularly!